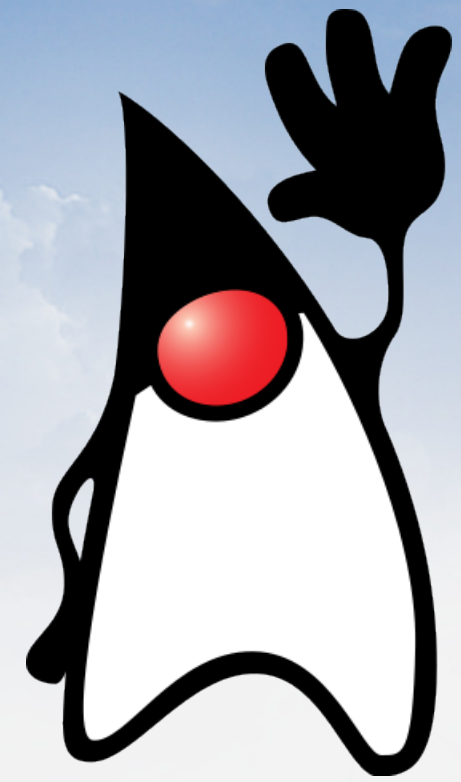




Inheritance Part 1

COP 3330
Department of Computer Science
University of Central Florida
Dr. Bacanli

What is inheritance?

- All of us inherit qualities from our parents.
 - Genes
 - Qualities
 - Etc...
- Object-Oriented Languages allows classes to follow this behavior in implementation.
- Inheritance is when a new class is created by acquiring an existing class's members and possibly embellishing them with new or modified capabilities.

How does this work in OOP?

- First lets understand some terminology.
- The newly created class we create comes from an existing class.
 - This existing class is known as the **super class** or **parent class**.
 - The newly created class is known as the **sub class** or **child class**.
 - Many languages use different terminologies.
 - C++ uses derived class as the child class and base class as the super class.
- Java only supports single inheritance, in which each class is derived from EXACTLY one super class.
- The child class inherits ALL attributes and ALL methods that are defined in the class.

Inheritance example

```
package carEx;
public class Car extends Vehicle{
    private String brandName;

    public Car(String bname){
        super();
        brandName = bname;
        System.out.println("Car is created");
    }

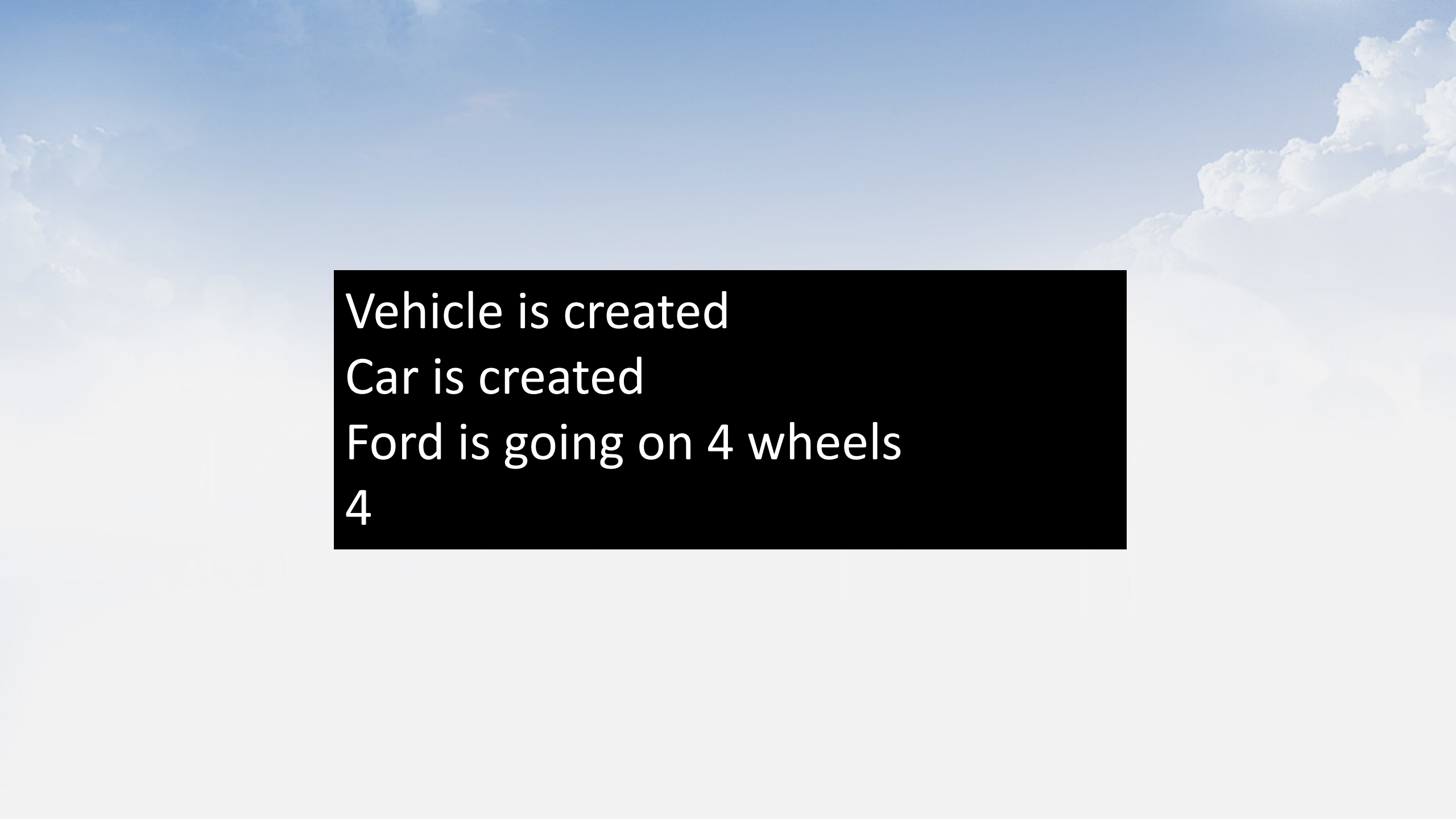
    public void go(){
        System.out.println(brandName+" is
going on " +getWheelNum()+" wheels");
    }
}
```

```
public void go(){
    System.out.println(brandName+
    " is going on " + super.getWheelNum() +
    " wheels");
}
```

```
package carEx;
public class Vehicle{
    private int wheelNumber;

    public Vehicle(){
        wheelNumber = 4;
        System.out.println("Vehicle is created");
    }
    public int getWheelNum(){
        return wheelNumber;
    }
}
```

```
package carEx;
public class vehiclesTester{
    public static void main(String[] args){
        Car myCar = new Car("Ford");
        myCar.go();
        System.out.println(myCar.getWheelNum());
    }
}
```



Vehicle is created

Car is created

Ford is going on 4 wheels

4

Method overriding

- Method overriding is when we have defined methods with the same and same signature
- When we create the the same parent method in the child class, we are using “method overriding”
- methods should have `@Override` before definition
 - Not obligatory
- Can we override private method of the parent class?
- Do private methods inherit?

Method overriding example

```
package carEx;
public class Car extends Vehicle{
    private String bname;

    public Car(String brandName){
        super();
        bname = brandName;
    }

    @Override
    public void go(){
        System.out.println(bname + " is going");
    }
}
```

//another overriding example

```
public void go(){
    super.go(); // method of super class
    System.out.println("Car is going");
}
```

```
package carEx;
public class Vehicle{
    private int wheelNumber;

    public Vehicle(){
        wheelNumber=4;
    }

    public void go(){
        System.out.println("Vehicle going");
    }
}
```

```
package carEx;
public class vehiclesTester{
    public static void main(String[] args){
        Car myCar = new Car("Ford");
        Vehicle veh = new Vehicle();
        veh.go();
        myCar.go();
    }
}
```

Method overriding rules

- The method name must not change
- The parameter list must not change: the overriding method must take the same number and type of parameters as the overridden method – otherwise, you would just be overloading the method.
- The return type must not change (Note: if the method returns an object, a subclass of that object is allowed as the return type).
- The access modifier must be either the same or a less restrictive one (for example, if the overridden method is protected, you can declare the overriding method as public, but not private).

Method overriding example

```
package carEx;
public class Car extends Vehicle{
    private String brandName;

    public Car(String bname){
        super();
        brandName = bname;
    }
    @Override
    public Car getSelf(){
        return this;
    }
}
```

If you remove the overridden method from the Car class, the code will still output true.

true

```
package carEx;
public class Vehicle{
    private int wheelNumber;

    public Vehicle(){
        wheelNumber = 4;
    }
    public Vehicle getSelf(){
        return this;
    }
}
```

```
package carEx;
public class VehiclesTester{
    public static void main(String[] args){
        Car myCar = new Car("Ford");
        Vehicle veh = new Vehicle();
        System.out.println(myCar.getSelf()==myCar);
    }
}
```

Method overloading

- Method overloading is when we have defined methods with the same name but different signature
- In fact we are creating a different method
 - We call it override because the name is same
- The method parameters must change: either the number or the type of parameters must be different in the two methods.
- The return type can be freely modified.
- The access modifier (public, private, and so on) can be freely modified.
- Can we overload private method of the parent class?

Method overloading example

```
package carEx;
public class Car extends Vehicle{
    private String brandName;

    public Car(String bname){
        super();
        brandName = bname;
    }

    protected void go(boolean isFlying){
        if(isFlying)
            System.out.println(brandName + " flies");
        else
            System.out.println(brandName + " goes");
    } //end of go method
}
```

Vehicle going
Ford flies

```
package carEx;
public class Vehicle{
    private int wheelNumber;

    public Vehicle(){
        wheelNumber = 4;
    }

    public void go(){
        System.out.println("Vehicle going");
    }
}
```

```
package carEx;
public class vehiclesTester{
    public static void main(String[] args){
        Car myCar = new Car("Ford");
        myCar.go();
        myCar.go(true);
    }
}
```

Overload/override as static and super in static

```
package carEx;
public class Car extends Vehicle{
    private String brandName;

    public Car(String bname){
        super();
        brandName = bname;
    }

    public static void go(int dummyVar){
        super.go();
    }

    public static void go(){
        super.go();
    }

    public static void main(String[] args){
        new Car();
    }
}
```

```
package carEx;
public class Vehicle{
    private int wheelNumber;

    public Vehicle(){
        wheelNumber = 4;
    }
    public void go(){
        System.out.println("Vehicle going");
    }
}
```

Cannot use super in a static context. Can't overload with static!

You can not override a method by setting it static.
"This static method cannot hide the instance method from Car.java"

We don't need a Car instance. Just constructor call.
Object is created and destroyed in main method.

final method and final class

- A final class cannot be extended by any other class
 - Your final class can not have children.
 - If all the constructors are private, the class will be indirectly final.
 - Why?
If the constructor is private, super() will not work!
 - Check Java API, there are some final classes
- A final method cannot be overridden
 - You can still overload them
 - They are still inherited.
 - The parent class has made a standard/perfect implementation. Child classes needs to accept that imposed standard.

Access Modifiers and statics

- All methods/variables except private are inherited.
- For static methods,
 - They can NOT be overridden
 - The parent static methods can still be called

static method inheritance example

```
package carEx;
public class Car extends Vehicle{
    private String brandName;

    public Car(String bname){
        super();
        brandName = bname;
    }

    public static void foo() {
        System.out.println("Car static");
    }
}
```

Car static
Car static
Vehicle static

```
package carEx;
public class Vehicle{
    private int wheelNumber;

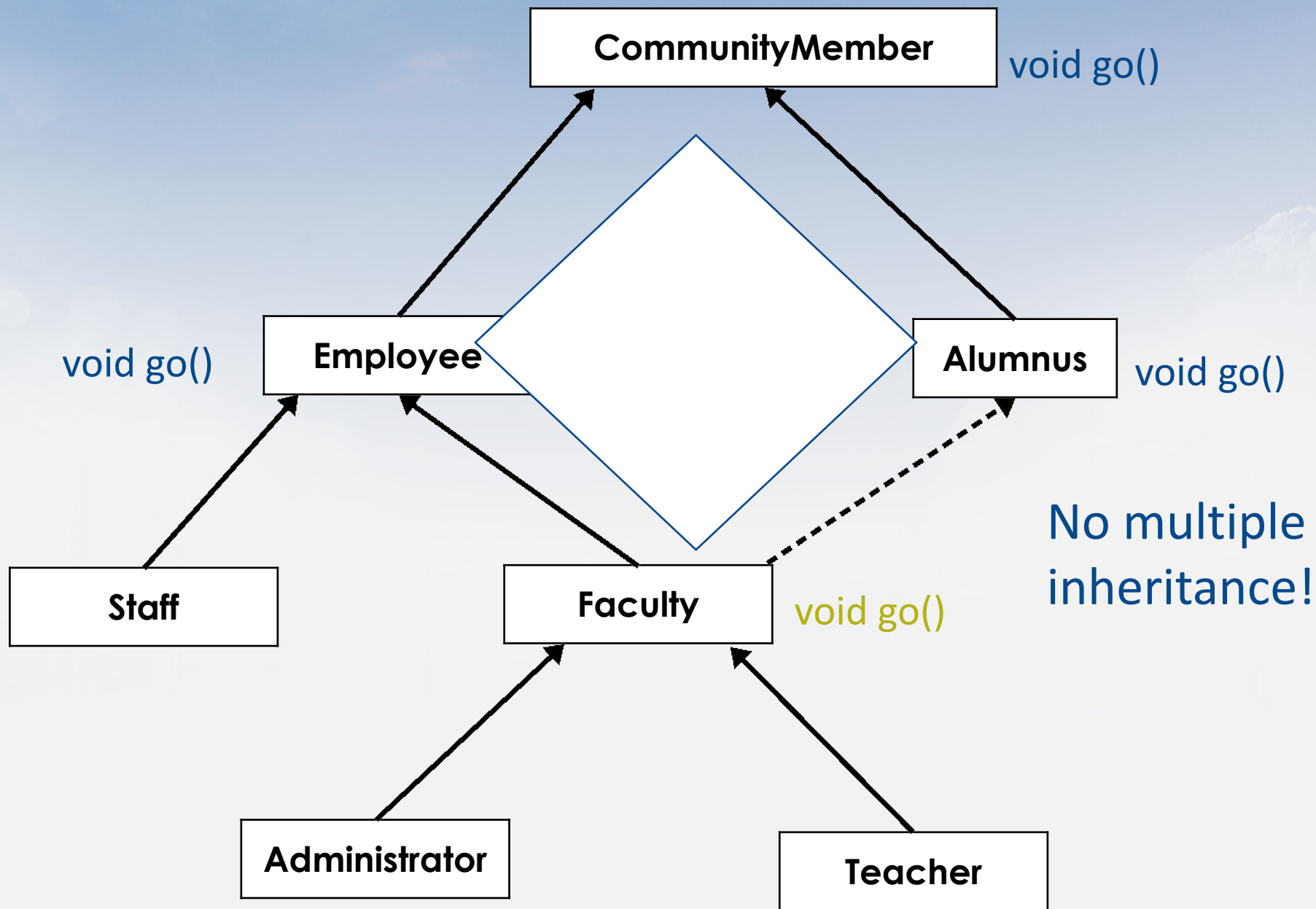
    public Vehicle(){
        wheelNumber = 4;
    }

    public static void foo() {
        System.out.println("Vehicle static");
    }
}
```

```
package carEx;
public class vehiclesTester{
    public static void main(String[] args){
        Car myCar = new Car(2);
        myCar.foo();
        Car.foo();
        Vehicle.foo(); //this will still work
    }
}
```

How can you determine if inheritance is even needed?

- Is-a relationship represents inheritance
 - Dog is a Animal
 - Bird is a Animal
 - Cat is a Animal
 - Student is a Human
 - Employee is a Human
- In fact it is more like **is a kind of** relationship, an object of a subclass can also be treated as an object of its super class.
 - If you feel like overriding methods will work nicely
- In a has-a relationship, an object contains as members references to other objects.
 - This we have seen already!



The Super Class of all of the classes

- All Classes are derived from the **Object** Class.
- Think about it?
 - How were we able to call the toString() method?
- Clone, equals, toString, hashCode etc.. are all inherited
 - We were overriding them
 - The toString() method of Object class is useless. It should be overridden.
 - ArrayList class has overridden toString(). That's why we can call it.

```
package carEx;
import java.util.ArrayList;
public class ArrayListTester{
    public static void main(String[] args){
        ArrayList<Integer> arr = new ArrayList<>();
        System.out.println(arr.toString());
    }
}
```

The Java API developers have overridden toString() method when they were implementing the ArrayList class.

toString

- We have seen this method on countless occasions now.
- String representation of the object
 - *brand year is 1999, Car is not runnable*
- Makes our job easy if we want to debug
- Always write toString() method for your class
- Make it always *public String toString()*
- DO NOT MAKE IT VOID!
 - What happens if you do?

Car class again

```
// Car.java
package testCars;

public class Car extends Object{

    private int year; //instance variable
    private String brand; //instance variable

    public Car(){
        super();
        year = 1999;
        brand = "Toyota";
    }
    @Override
    public String toString(){
        return "Brand is " + brand +
            " Year is " + year;
    }
}
```

when you don't write toString() inside print, java does that for you!

```
// CarTest.java
package testCars;

public class CarTest{

    public static void main(String[] args){
        Car mycar = new Car();
        System.out.println(mycar.toString());
        System.out.println(mycar);
    } //end of main
}
```

Brand is Toyota Year is 1999
Brand is Toyota Year is 1999